Свёрточные нейронные сети (часть 4)

Свёрточные нейронные сети (часть 4) Лекция 7

Седых И.А.

Минутка кода: размерность тензора

Прежде чем переходить к созданию свёрточного слоя, необходимо создать структуру для тензора. Тензор представляет из себя 3D массив. У тензора имеются три размерности, а значит нам потребуется структура для хранения размера тензора, мы назовём её TensorSize:

```
// размерность тензора
struct TensorSize {
    int depth; // глубина
    int height; // высота
    int width; // ширина
};
```

Минутка кода: класс Тензор - 1

Теперь создадим тензор. В нём нам потребуется хранить depth·height·width вещественных чисел, которые мы будем хранить в одномерном векторе для уменьшения количества обращений к элементам по индексам.

```
// тензор
class Tensor {
   TensorSize size; // размерность тензора
   std::vector<double> values; // значения тензора
   int dw; // произведение глубины на ширину для индексации
   void Init(int width, int height, int depth);

public:
   Tensor(int width, int height, int depth); // создание из размеров
   Tensor(const TensorSize &size); // создание из размера
   double& operator()(int d, int i, int j); // индексация
   double operator()(int d, int i, int j) const; // индексация
   TensorSize GetSize() const; // получение размера
    // вывод тензора
   friend std::ostream& operator<<(std::ostream& os, const Tensor &tensor);
};
```

Минутка кода: класс Тензор - 2

```
// инициализация по размерам
void Tensor::Init(int width, int height, int depth) {
    size.width = width; // запоминаем ширину
    size.height = height; // запоминаем высоту
    size.depth = depth; // запоминаем глубину

    // запоминаем произведение глубины на ширину для индексации
    dw = depth * width;

    values = std::vector<double>(width * height * depth, 0); // создаём вектор из
                                                             // width * height * depth нулей
}

// создание из размеров
Tensor::Tensor(int width, int height, int depth) {
    Init(width, height, depth);
}

// создание из размера
Tensor::Tensor(const TensorSize &size) {
    Init(size.width, size.height, size.depth);
}
```

Минутка кода: класс Тензор - 3

```
// индексация
double& Tensor::operator()(int d, int i, int j) {
    return values[i * dw + j * size.depth + d];
}
double Tensor::operator()(int d, int i, int j) const {
    return values[i * dw + j * size.depth + d];
}
// получение размера
TensorSize Tensor::GetSize() const {
    return size;
}
// вывод тензора
std::ostream& operator<<(std::ostream& os, const Tensor &tensor) {
    for (int d = 0; d < tensor.size.depth; d++) {
        for (int i = 0; i < tensor.size.height; i++) {
            for (int j = 0; j < tensor.size.width; j++)
                os << tensor.values[i * tensor.dw + j * tensor.size.depth + d] << " ";
                os << std::endl;
                }
            os << std::endl;
        }
return os;
}
```

Минутка кода: класс Свёрточный слой - 1

Свёрточный слой содержит в себе набор фильтров и смещений. Также для создания слоя необходимо задать несколько гиперпараметров, : fs — **размер фильтров**, fc — **количество фильтров**, S — **шаг свёртки** и P — **дополнение нулями**. Поскольку фильтры являются тензорами, а смещения всего лишь числами, то добавим фильтры в виде вектора тензоров и назовём W, а смещения добавим в виде векторов вещественных чисел и назовём их b. Поскольку эти веса являются обучаемыми, нам потребуется аналогичные наборы векторов для хранения градиентов.

```
class ConvLayer {
    std::default_random_engine generator;  // генератор случайных чисел
    std::normal_distribution<double> distribution;  // с нормальным распределением

    TensorSize inputSize; // размер входа
    TensorSize outputSize; // размер выхода

    std::vector<Tensor> W; // фильтры
    std::vector<double> b; // смещения

    std::vector<Tensor> dW; // градиенты фильтров
    std::vector<double> db; // градиенты смещений

    int P; // дополнение нулями
    int S; // шаг свёртки
```

Минутка кода: класс Свёрточный слой - 2

```
    int fc; // количество фильтров
    int fs; // размер фильтров
    int fd; // глубина фильтров

void InitWeights(); // инициализация весовых коэффициентов

public:
    ConvLayer(TensorSize size, int fc, int fs, int P, int S); // создание слоя

    Tensor Forward(const Tensor &X); // прямое распространение сигнала

    // обратное распространение ошибки
    Tensor Backward(const Tensor &dout, const Tensor &X);
    void UpdateWeights(double learningRate); // обновление весовых коэффициентов
};
```

Само создание слоя довольно простое: необходимо запомнить переданные параметры в полях класса и инициализировать весовые коэффициенты фильтров и смещений.

Минутка кода: класс Свёрточный слой - 3

```
// создание свёрточного слоя
ConvLayer::ConvLayer(TensorSize size, int fc, int fs, int P, int S) : distribution(0.0, sqrt(2.0 / (fs*fs*size.depth))) {

    // запоминаем входной размер
    inputSize.width = size.width;
    inputSize.height = size.height;
    inputSize.depth = size.depth;

    // вычисляем выходной размер
    outputSize.width = (size.width - fs + 2 * P) / S + 1;
    outputSize.height = (size.height - fs + 2 * P) / S + 1;
    outputSize.depth = fc;

    this->P = P; // сохраняем дополнение нулями
    this->S = S; // сохраняем шаг свёртки

    this->fc = fc; // сохраняем число фильтров
    this->fs = fs; // сохраняем размер фильтров
    this->fd = size.depth; // сохраняем глубину фильтров
```

Минутка кода: класс Свёрточный слой - 4

```
    // добавляем fc тензоров для весов фильтров и их градиентов
    W = std::vector<Tensor>(fc, Tensor(fs, fs, fd));
    dW = std::vector<Tensor>(fc, Tensor(fs, fs, fd));

    // добавляем fc нулей для весов смещения и их градиентов
    b = std::vector<double>(fc, 0);
    db = std::vector<double>(fc, 0);

    InitWeights(); // инициализируем весовые коэффициенты
}
```

Минутка кода: класс Свёрточный слой - 5

Для заполнения весовых коэффициентов случайными числами просто проходимся по всем тензорам и присваиваем каждому элементу очередное случайное число:

```
// инициализация весовых коэффициентов
void ConvLayer::InitWeights() {
   // проходимся по каждому из фильтров
   for (int index = 0; index < fc; index++) {
      for (int i = 0; i < fs; i++)
         for (int j = 0; j < fs; j++)
            for (int k = 0; k < fd; k++)
               // генерируем случайное число и записываем его в элемент фильтра
               W[index](k, i, j) = distribution(generator);

      b[index] = 0.01; // все смещения устанавливаем в 0.01
    }
}
```

Минутка кода: прямое распространение сигнала - 1

```
Tensor ConvLayer::Forward(const Tensor &X) {
   Tensor output(outputSize); // создаём выходной тензор

   // проходимся по каждому из фильтров
   for (int f = 0; f < fc; f++) {
      for (int y = 0; y < outputSize.height; y++) {
         for (int x = 0; x < outputSize.width; x++) {
            double sum = b[f]; // сразу прибавляем смещение

            // проходимся по фильтрам
            for (int i = 0; i < fs; i++) {
               for (int j = 0; j < fs; j++) {
                  int i0 = S * y + i - P;
                  int j0 = S * x + j - P;
                  // поскольку вне границ входного тензора элементы нулевые,
                  // то просто игнорируем их
                  if (i0 < 0 || i0 >= inputSize.height || j0 < 0 || j0 >= inputSize.width)
                     continue;
                  // проходимся по всей глубине тензора и считаем сумму
                  for (int c = 0; c < fd; c++)
                     sum += X(c, i0, j0) * W[f](c, i, j);
               }
            }
            output(f, y, x) = sum; // записываем результат свёртки в выходной тензор
         }
      }
   }
   return output; // возвращаем выходной тензор
}
```